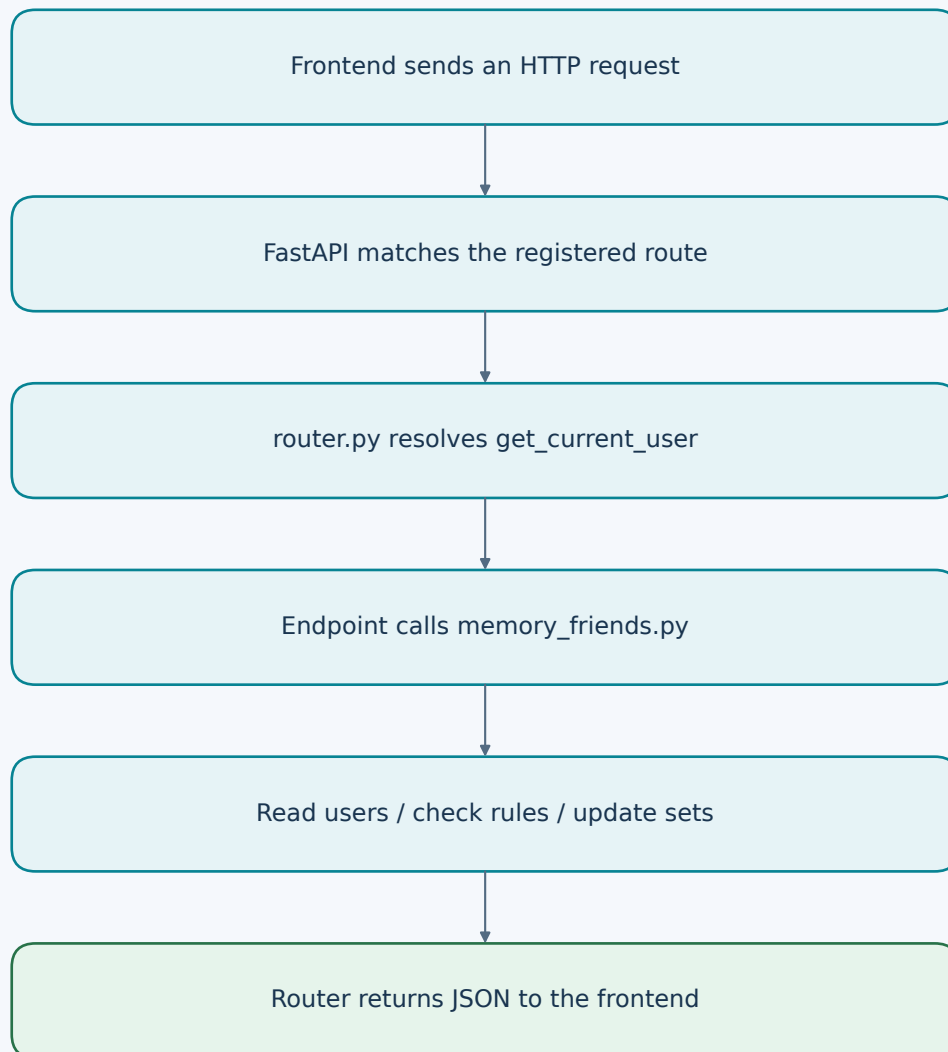


1. How the system is connected

Request path / current two-file implementation

The router handles HTTP and authentication. The memory module checks friendship rules and stores relationships. This PDF describes the files currently on disk, not the earlier refactored design.



Who owns what?

Authentication: accounts, passwords, tokens

Friendship: pending requests and accepted pairs

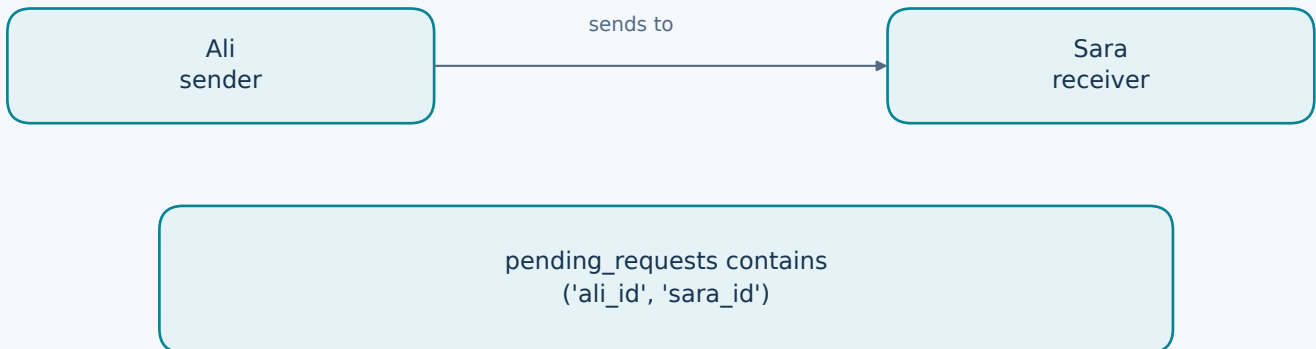
`main.py` registers `friends_router` at startup. All seven endpoints require authentication. This version has no database storage or live notifications.

2. Understand the two stores

Directed requests / shared friendships

A request has a sender and a receiver. A friendship is one shared pair. A user may appear in many pairs and can have many friends.

PENDING REQUEST



ACCEPTED FRIENDSHIP



One pair per friendship

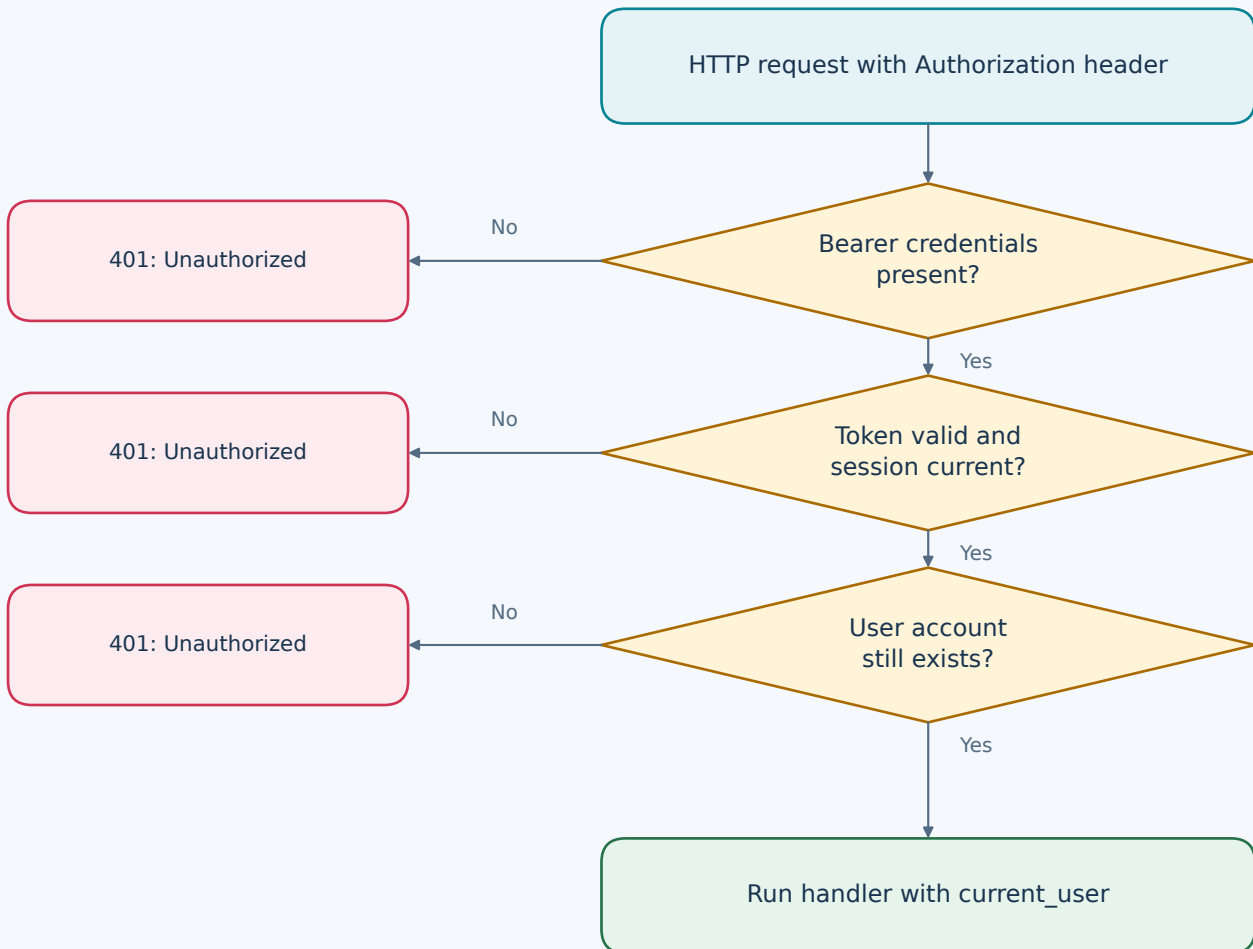
Ali + Sara, Ali + Omar, Ali + Lina = 3 friends for Ali

Tuples preserve sender/receiver order. A frozenset ignores order, so Ali-Sara and Sara-Ali are the same friendship. These readable IDs stand in for actual account IDs.

3. Authenticate every request

Shared dependency / `get_current_user`

The acting user comes from the bearer token. The ID in a friendship URL identifies the other participant; it cannot select who is logged in.



Example

```

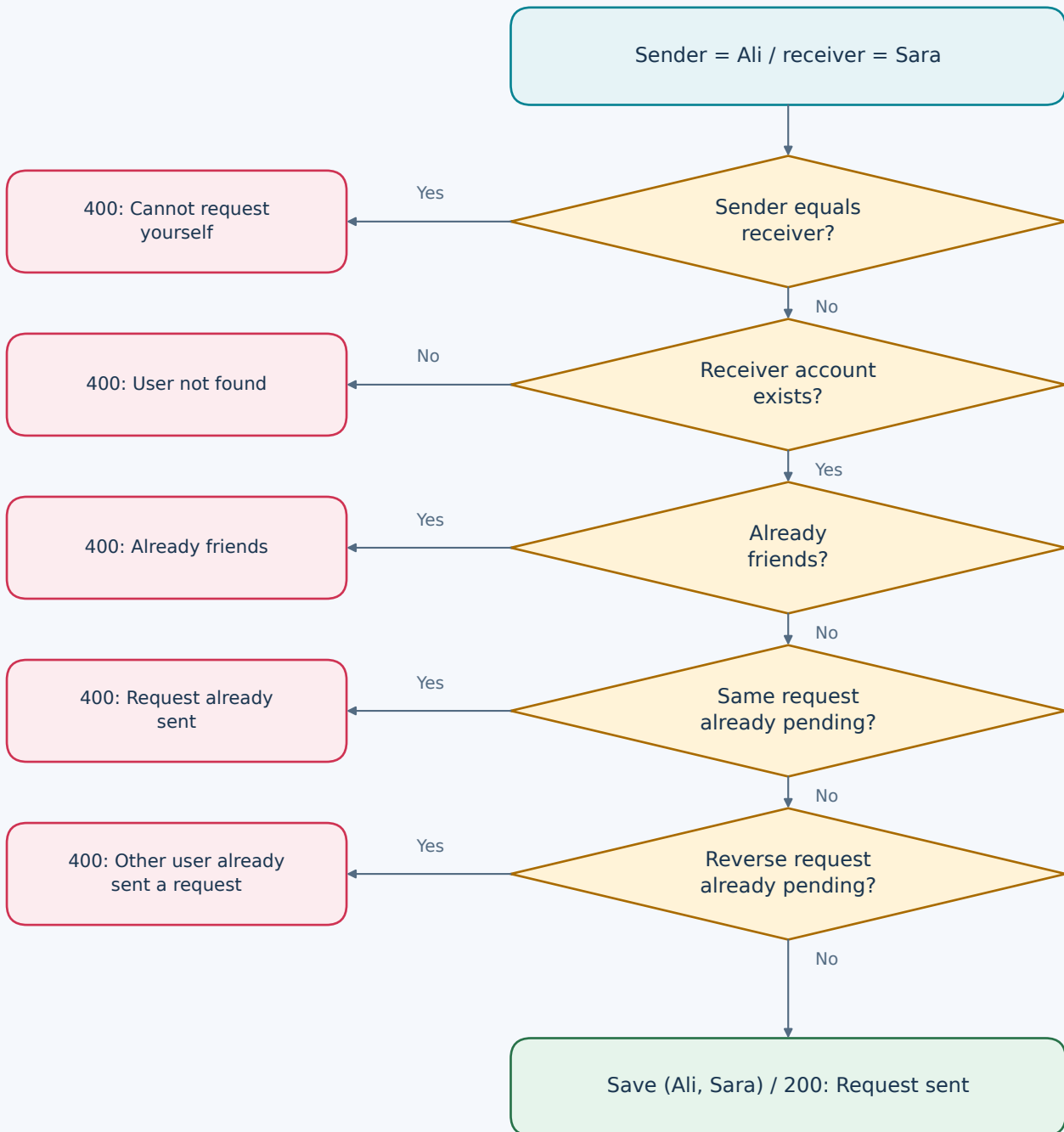
Authorization: Bearer <access_token>
Actor ID = current_user['id']
  
```

The security module checks signature, expiry, required claims and account/session validity. Friendship handlers run only after authentication succeeds.

4. Send a friend request

POST /friends/request/{user_id}

Ali is logged in and supplies Sara's ID in the URL. `send_friend()` checks every rule before adding the directed request.



Example

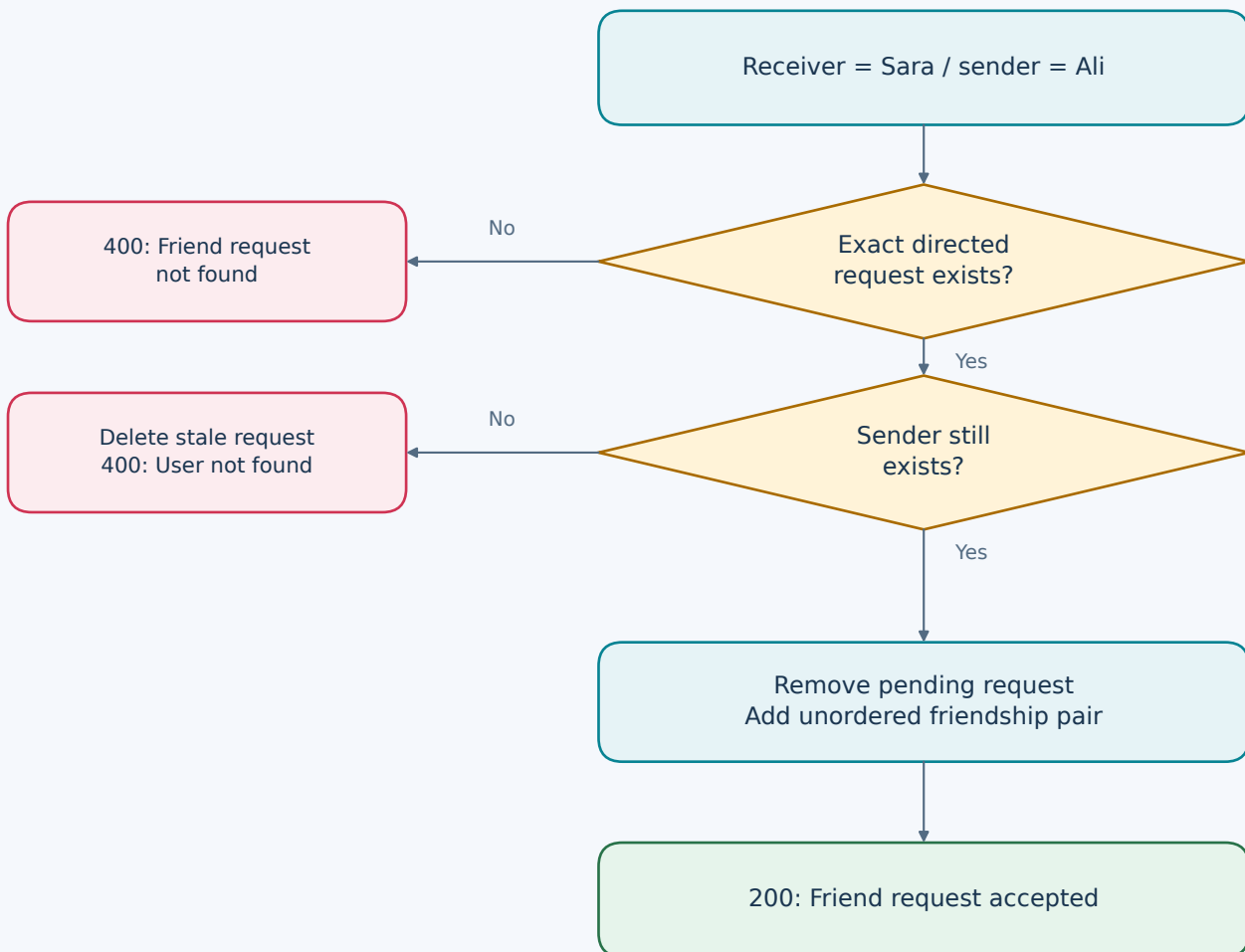
```
POST /friends/request/<sara_id>
```

Use Ali's token. A reverse pending request is rejected; sending it does not automatically accept the first request.

5. Accept a friend request

POST /friends/accept/{sender_id}

Sara is logged in and supplies Ali's ID. The lookup uses (sender_id, receiver_id), even though accept_request() takes the receiver first.



Example

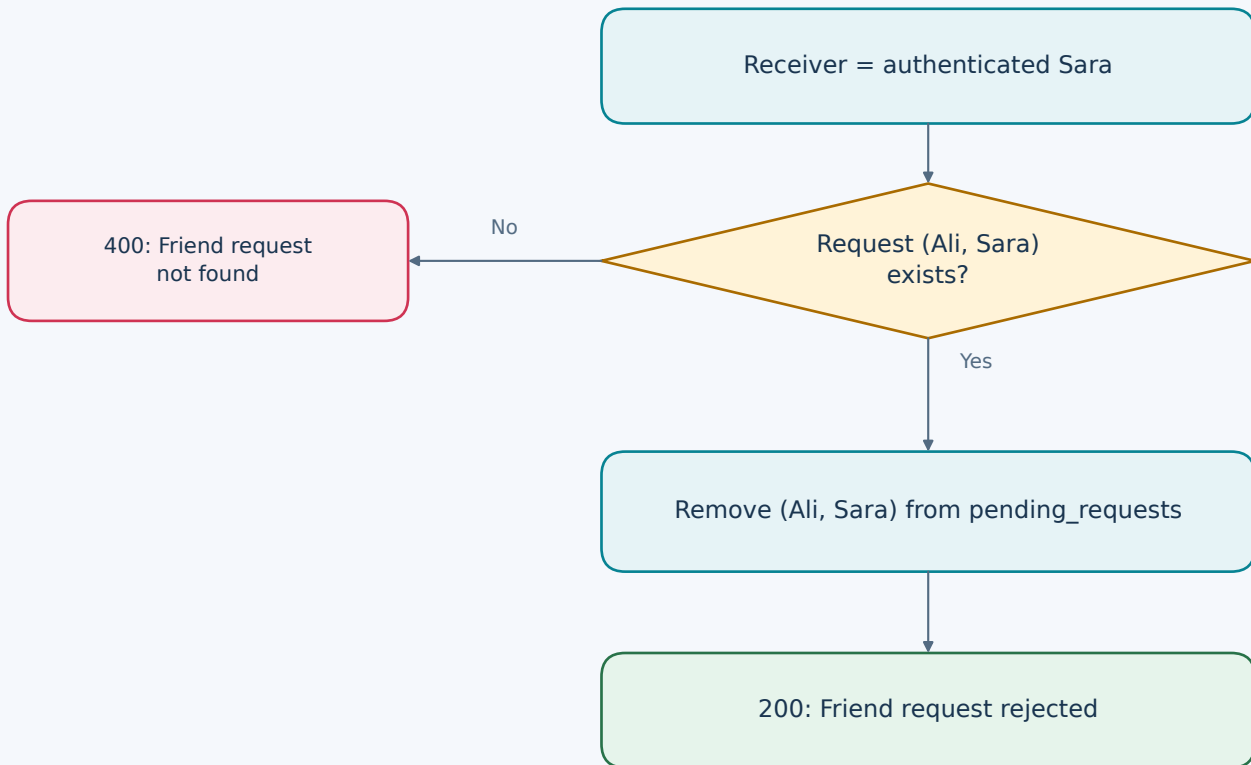
```
POST /friends/accept/<ali_id>
Use Sara's token, not Ali's token.
```

Both users now see each other as friends. A second acceptance fails because the pending request is gone. Current code cleans only this stale request when the sender has disappeared.

6. Reject a received request

POST /friends/reject/{sender_id}

Sara declines Ali's invitation. Only the receiver can reject the matching incoming request. Rejecting does not create a friendship.



Example

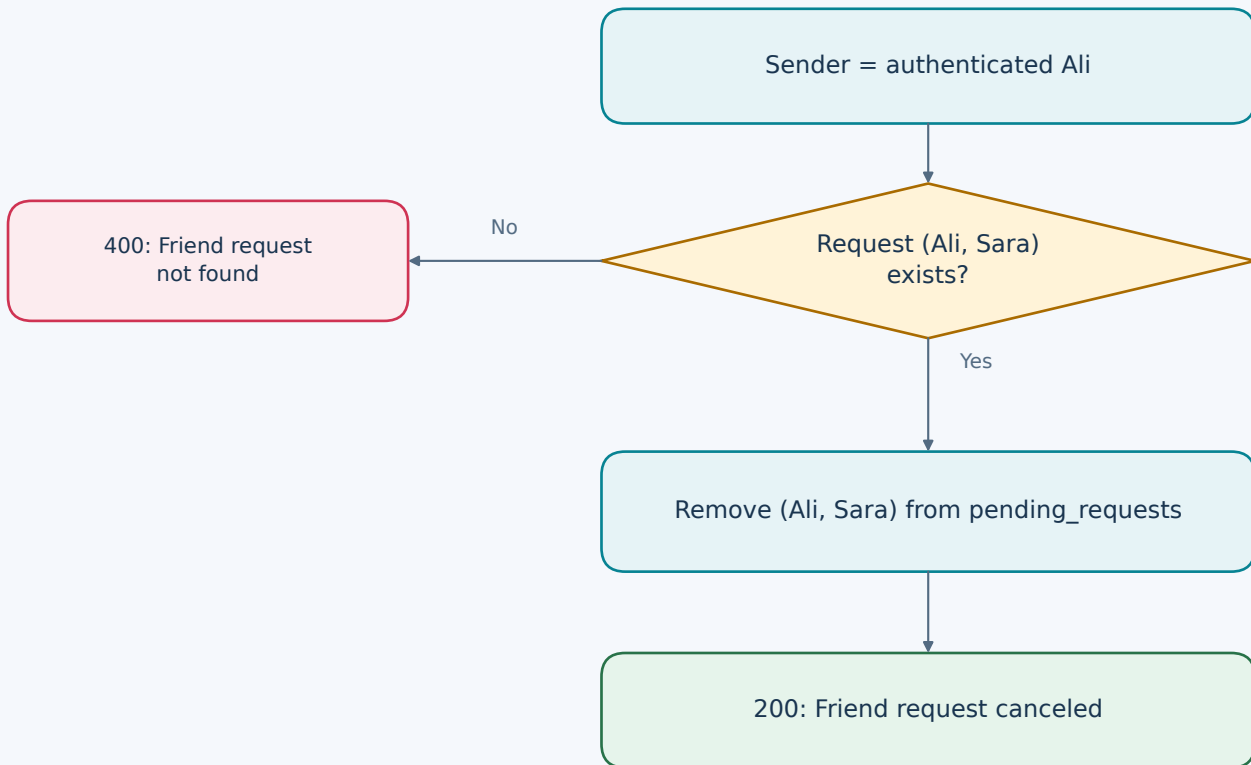
```
POST /friends/reject/<ali_id>
Use Sara's token.
```

Repeating this operation returns 400 because the request was removed. Rejection is not stored as history; a new request may be sent later.

7. Cancel a sent request

DELETE /friends/request/{receiver_id}

Ali withdraws the request he sent to Sara. The operation removes the same tuple as rejection, but the acting user must be the sender.



Example

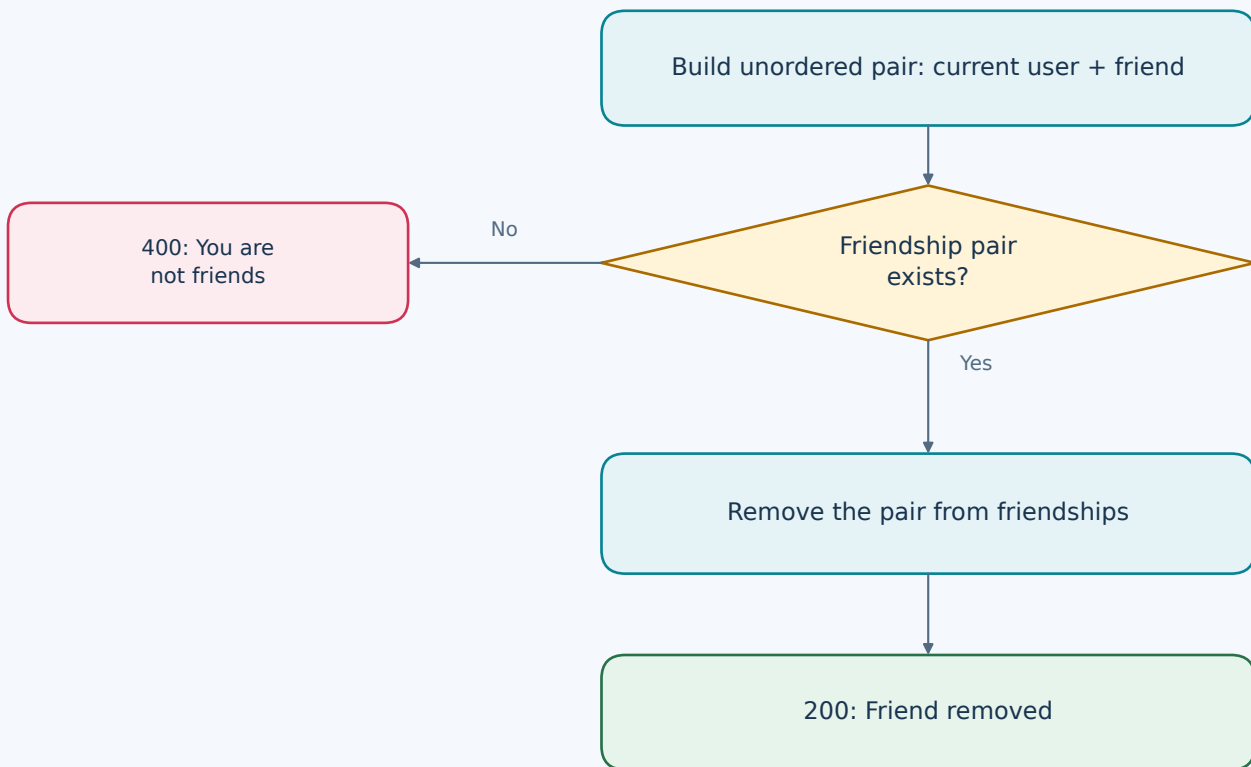
```
DELETE /friends/request/<sara_id>
Use Ali's token.
```

Sara can reject; Ali can cancel. Once accepted, this request no longer exists. Use the remove-friend endpoint to end an accepted friendship.

8. Remove an accepted friend

DELETE /friends/{friend_id}

Either participant can remove the shared friendship. The router uses `memory_friends.remove_friend()` to avoid calling its own handler by mistake.



Example

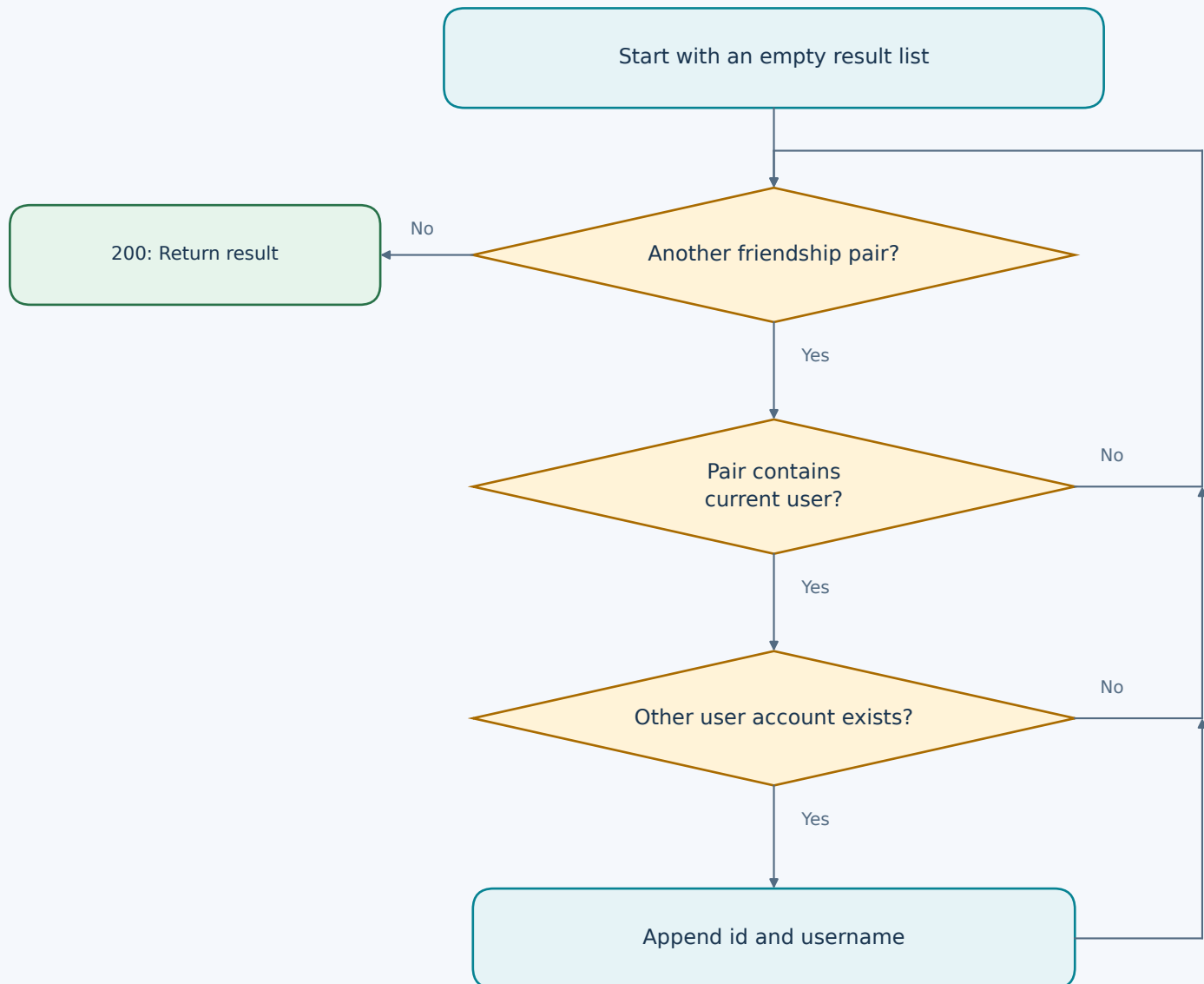
```
DELETE /friends/<sara_id>
Use Ali's token; Sara may also remove Ali.
```

Deleting one pair removes the relationship from both lists. Other friendships stay intact. An unrelated user cannot address a pair that does not contain their own authenticated ID.

9. List accepted friends

GET /friends/

Find every friendship pair containing the logged-in user, then look up the other participant.
Missing accounts are skipped.



Example response

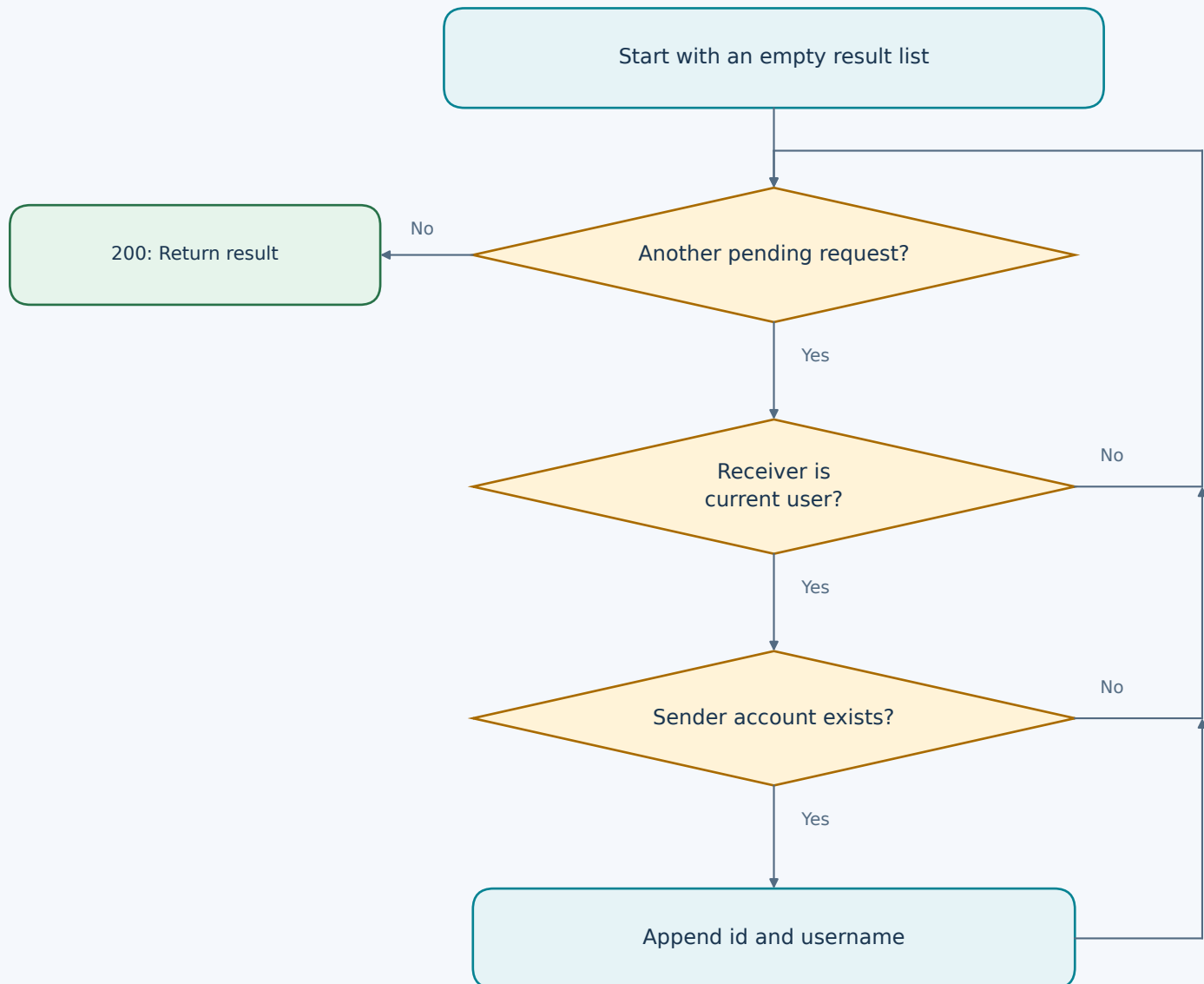
```
[{"id": "<sara_id>", "username": "Sara"}]
```

The current response has no guaranteed order. It includes neither passwords nor email addresses. An empty list is a successful result, not an error.

10. List incoming requests

GET /friends/requests

Return sender summaries only for requests addressed to the logged-in user. Missing sender accounts are skipped.



Example response

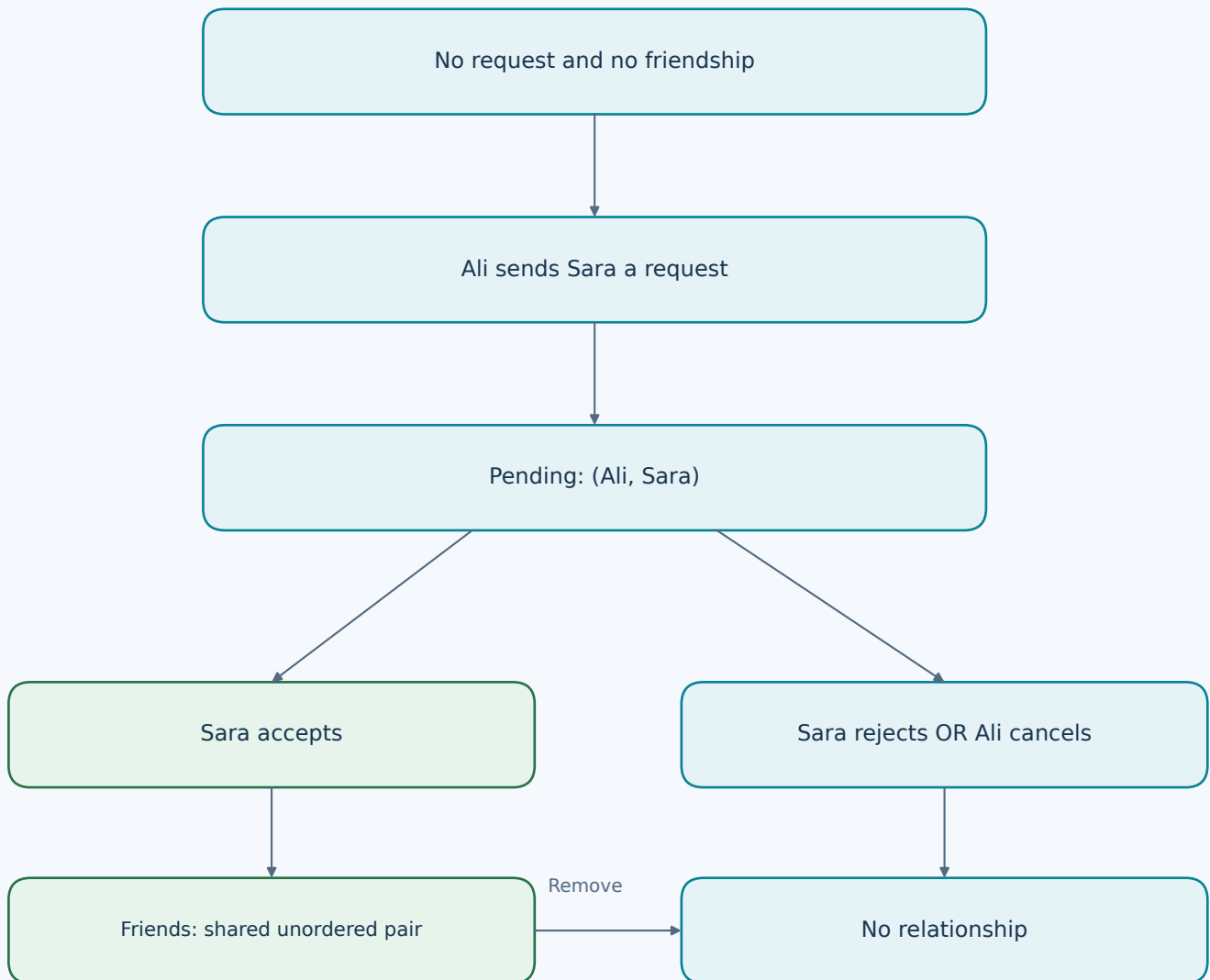
```
[{"id": "<ali_id>", "username": "Ali"}]
```

The current response has no guaranteed order. It includes neither passwords nor email addresses. An empty list is a successful result, not an error.

11. Follow the complete lifecycle

Ali and Sara / state changes

Sending is only an invitation. Acceptance creates the friendship. Rejection, cancellation and removal return the pair to having no relationship.



Check both accounts

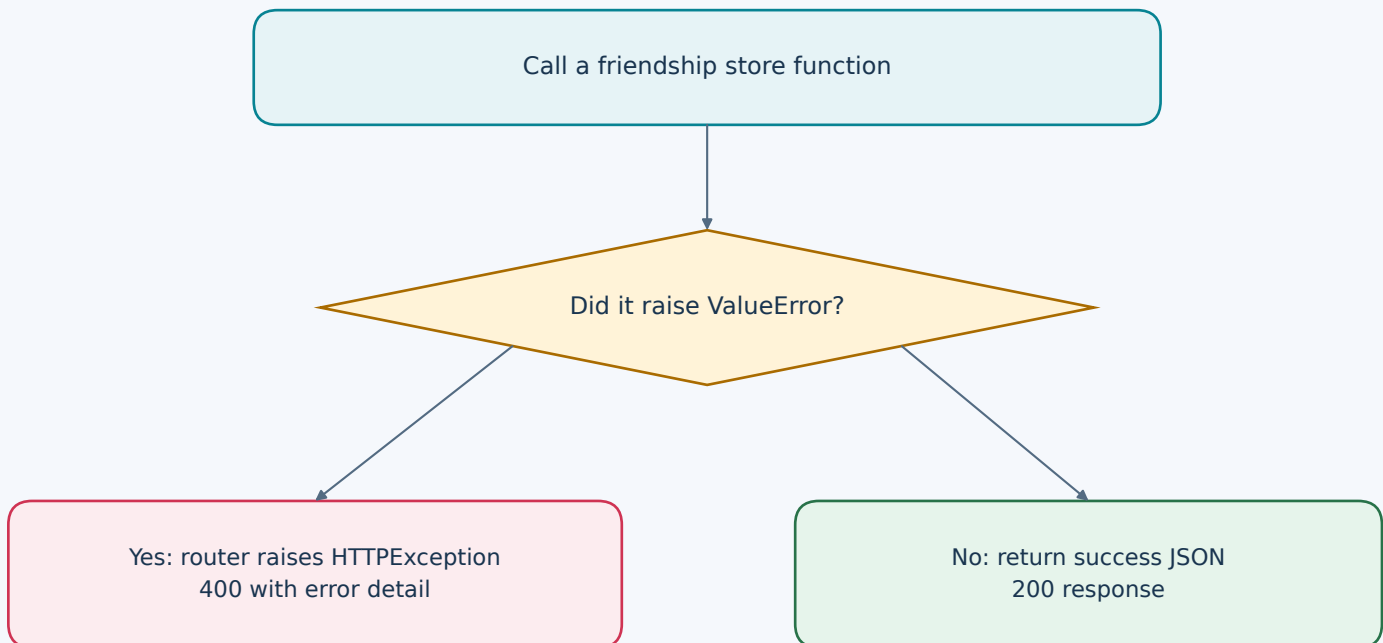
After acceptance: Ali lists Sara; Sara lists Ali.
After removal: neither lists the other.

No live notification is sent by this code. Users fetch the lists themselves. Restarting the server clears the temporary users and relationship data.

12. Errors, permissions and limits

Current behavior / what to verify

The router catches `ValueError` from the store and converts it to HTTP 400. Authentication errors are handled separately. Current URL IDs are strings, not UUID-validated parameters.



TEST THESE CASES

- Self-request, unknown user, duplicate or reverse pending request.
- Accept, reject or cancel twice; remove someone who is not a friend.
- A third user attempts to accept or remove another pair.
- Missing, invalid, expired or revoked token: expect 401.
- Multiple friends: removing one must not affect the others.

Current limitations

Memory only: no persistence or shared multi-worker state.
No outgoing-request list, timestamps, or response models.

Unexpected failures are not converted to 400. Database migration will need transactions, uniqueness constraints and account-deletion cleanup. This PDF is documentation, not a fresh API test run.